

1. INTRODUÇÃO

Os sistemas operacionais são responsáveis por gerenciar os recursos computacionais e coordenar a execução dos processos, garantindo o funcionamento eficiente e organizado do computador. Para cumprir esse papel, desempenham diversas funções essenciais, dentre as quais se destaca o escalonamento de processos. Esse mecanismo é responsável por determinar qual processo utilizará a Unidade Central de Processamento (CPU) em cada instante, buscando distribuir os recursos de forma eficiente e equilibrada. Em sistemas modernos, essa decisão pode levar em consideração diferentes critérios de escolha, tais como a prioridade do processo, o tempo que está na fila de pronto e a utilização de recursos.

Nesse contexto, o sistema operacional educacional XV6, desenvolvido pelo *Massachusetts Institute of Technology* (MIT) e inspirado nas primeiras versões do UNIX, apresenta-se como uma plataforma voltada ao estudo e à implementação dos principais conceitos de sistemas operacionais, permitindo a modificação de seu kernel para fins didáticos e experimentais. Em sua configuração padrão, o XV6 emprega o algoritmo de escalonamento *Round-Robin*, no qual todos os processos recebem fatias de tempo (*quantum*) de mesma duração e são tratados com igual prioridade. Apesar de ser suficiente para fins didáticos, esse modelo não representa a complexidade dos sistemas reais, que utilizam políticas de escalonamento mais sofisticadas.

A motivação deste projeto é justamente aproximar o XV6 de um cenário mais realista, permitindo diferenciar processos com base em níveis de prioridade. Essa modificação possibilita um melhor controle do uso da CPU e a exploração, na prática, de como o kernel gerencia processos sob conceitos fundamentais de escalonamento e concorrência. Além disso, o objetivo dessa abordagem é evitar o problema de inanição (*starvation*), garantindo que todos os processos eventualmente recebam tempo de CPU, mesmo aqueles que possuem uma prioridade inicial baixa.

Neste contexto, este relatório documenta a implementação de um mecanismo de prioridade de processos no sistema operacional XV6. O foco principal consistiu na implementação da chamada de sistema `getpriority()`, que permite a consulta da prioridade de um processo por meio de seu identificador (PID). Para isso, foram efetuadas diversas modificações estruturais no kernel, incluindo a definição das novas syscalls, seus registros na tabela de chamadas de sistema e a implementação da lógica necessária para o gerenciamento das prioridades.

2. CÓDIGO

2.1 KERNEL / PROC.H - ADIÇÃO DE CAMPOS NA STRUCT PROC

Para viabilizar a implementação de um escalonamento por prioridade e o mecanismo de envelhecimento (*aging*), foi adicionado dois campos à estrutura do processo : `int priority`, armazena a prioridade atual do processo, e `int ciclocpu` que contabiliza o tempo de permanência do processo na fila de espera

2.2 ALLOCPROC () em KERNEL/PROC.C - INICIALIZAÇÃO

Na inicialização do processo, os novos atributos são iniciados com valores predefinidos: `priority` recebe um valor neutro de prioridade e `ciclocpu` é zerado.

2.3 GET PRIORITY(INT PID) em KERNEL/PROC.C

Percorre o array global de processos para localizar o processo com o PID informado, e retorna sua prioridade (-1 caso não encontrar). A função é alocada neste arquivo - não em `sysproc.c` - para respeitar a convenção estrutural xv6, que exige manter todo acesso à tabela de processos encapsulada neste local.

2.4 UPDATE_AGING() em KERNEL/PROC.C - MECANISMO DE AGING

A cada vez que esta função é chamada, ela itera sobre o vetor de processos, e incrementa o contador `ciclocpu` de todos aqueles que se encontram em estado de espera (*RUNNABLE*). Quando o contador atingir 20, a sua prioridade numérica desce em 1, aumentando a prioridade efetiva do processo, e o contador zera para recomençar a contagem. Este mecanismo é ponto crítico na prevenção da inanição (*starvation*). Ele assegura que processos, inicialmente com baixa prioridade, ganhem preferência ao longo do tempo, garantindo que eventualmente possuam prioridade suficiente para serem despachados pelo escalonador.

2.5 SCHEDULER () em KERNEL/PROC.C - REESCRITO

Esta modificação representa a alteração arquitetural central no sistema, sendo responsável por substituir a política padrão *Round-Robin* do XV6 por um algoritmo de escalonamento baseado em prioridades com mecanismo de envelhecimento (*aging*). Na nova rotina, a função `update_aging()` é invocada a cada iteração do laço principal do escalonador, garantindo o processamento contínuo da fila de processos em espera.

Durante essa execução, o algoritmo realiza uma varredura única pelo vetor de processos. O objetivo é avaliar todos os processos em estado de prontidão (*RUNNABLE*) para selecionar aquele que detém a maior prioridade efetiva, que é representada pelo menor valor numérico.

Para maximizar a concorrência, o sistema emprega um gerenciamento dinâmico e otimizado de *locks*. O bloqueio é mantido ativo exclusivamente para o melhor candidato parcial encontrado durante a varredura. Caso o iterador identifique um processo com prioridade superior, o escalonador libera imediatamente o *lock* do candidato anterior e adquire o bloqueio do novo processo escolhido.

2.6 USER/AGINGTEST.C e USER/GETPRITEST.C - REALIZAÇÃO DE TESTES

Para validar a implementação, foram desenvolvidos programas de teste que simulam cenários de concorrência real gerando múltiplos processos através da chamada *fork()*.

- *getpritest.c*: teste básico de validação. Serve para confirmar, de forma direta, se a chamada de sistema *getpriority()* está funcionando e retornando os valores corretos.
- *agingtest.c*: teste de carga e concorrência. Foi desenvolvido especificamente para avaliar se o mecanismo de aging se comporta conforme o esperado quando existem vários processos competindo simultaneamente pela CPU.

3. RESULTADOS

3.1 METODOLOGIA

Comando *agingtest* foi executado três vezes em sequência, na mesma sessão do QEMU, com apenas um núcleo, sem reiniciar o sistema entre as chamadas. Os dados apresentados a seguir foram estruturados a partir do cruzamento dos tempos de execução (duração) com os registros capturados na saída do console, o que permitiu isolar e analisar cada linha individualmente.

3.2 EXECUÇÕES

Figura 1 - Visualização da execução do primeiro teste.

filho	pid	start	end	duração	prioridade_final
0	6	159	161	2	10
1	7	161	162	1	9
2	8	162	169	7	3
3	9	163	171	8	0
4	10	164	172	8	0
5	11	165	171	6	0
6	12	166	174	8	0
7	13	167	174	7	0

Figura 2 - Visualização da execução do segundo teste.

filho	pid	start	end	duração	prioridade_final
0	15	358	361	3	10
1	16	361	362	1	9
2	17	362	369	7	3
3	18	363	369	6	2
4	19	364	371	7	0
5	20	365	372	7	0
6	21	366	374	8	0
7	22	367	374	7	0

Figura 3 - Visualização da execução do terceiro teste.

filho	pid	start	end	duração	prioridade_final
0	24	479	482	3	10
1	25	482	483	1	9
2	26	483	490	7	3
3	27	484	490	6	2
4	28	485	492	7	0
5	29	486	493	7	0
6	30	487	495	8	0
7	31	488	495	7	0

4. ANÁLISE

A análise preliminar dos dados obtidos revela que o sistema operacional XV6 modificado abandonou seu comportamento nativo puramente *Round-Robin*, passando a atuar de acordo com a expectativa teórica do projeto. Observa-se uma alta consistência entre as execuções independentes, o que atua como evidência de que o comportamento do novo *scheduler* é determinístico, e não fruto de ruídos ou flutuações casuais de processamento. Esse padrão reforça que a lógica de implementação do mecanismo de *aging* responde de forma previsível e repetível a uma mesma carga de trabalho, produzindo a mesma ordem relativa de prioridades sempre que o teste é executado sob condições idênticas. Consequentemente, atesta-se que a alocação da CPU passou a respeitar rigorosamente as diretrizes de escalonamento prioritário estabelecidas no *kernel*, validando a arquitetura proposta.

Ademais, ao examinar os dados das três execuções, constata-se que a prioridade final efetiva dos processos aumenta (representada por uma sequência numérica decrescente) em relação à sua ordem de criação. Esse comportamento corrobora integralmente a lógica de envelhecimento implementada. Como processos instanciados posteriormente tendem a passar mais tempo aguardando na fila de prontidão — visto que os processos mais antigos já estão disputando ou ocupando a CPU —, eles acumulam sucessivos ciclos de espera em seus respectivos contadores. Consequentemente, atingem o limiar de alteração mais rapidamente, tendo seu valor numérico decrementado. Essa dinâmica comprova a eficácia do sistema na prevenção ativa da inanição (*starvation*), assegurando que nenhum processo seja postergado indefinidamente pelo escalonador.

5. CONCLUSÃO

Em síntese, a implementação dos códigos desenvolvidos nesse projeto modificou o sistema de escalonamento Round-Robin para um mecanismo de escalonamento baseado em prioridade, necessário para a implementação do *aging* e tornando útil a criação da chamada de sistema para pegar a prioridade do processo. Para isso, foram realizadas modificações estruturais no kernel adicionando novos campos à estrutura de processos e a implementação da chamada de sistema *getpriority()*. Além disso, foi desenvolvido o mecanismo de envelhecimento (*aging*) para garantir que processos de baixa prioridade não fiquem indefinidamente sem execução e programas de teste para validar a implementação do *aging*.

Os resultados obtidos durante os testes demonstraram que as alterações produziram o comportamento esperado. A chamada de sistema implementada apresentou funcionamento correto na consulta das prioridades dos processos, enquanto o novo escalonador passou a selecionar os processos de acordo com a prioridade. Paralelamente, o mecanismo de aging mostrou-se eficaz ao promover, de forma gradual, o aumento da prioridade dos processos que permaneceram em espera, assegurando a prevenção da inanição (starvation).

Portanto, o desenvolvimento deste projeto proporcionou uma compreensão mais aprofundada sobre a organização interna do kernel do XV6, o funcionamento das chamadas de sistema e os desafios envolvidos na implementação de políticas de escalonamento. Dessa forma, conclui-se que a solução desenvolvida atingiu os resultados esperados, confirmando que o sistema atende aos objetivos de implementar a syscall para consultar prioridade e evitar starvation por meio do aging, definidos inicialmente para o projeto.

REFERÊNCIAS

SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. **Operating System Concepts**. 10. ed. Hoboken: Wiley, 2018. cap. 5.

ARPACI-DUSSEAU, Remzi H.; ARPACI-DUSSEAU, Andrea C. **Operating Systems: Three Easy Pieces**. 1. ed. [S. l.]: CreateSpace Independent Publishing Platform, 2018. cap. 7.

COX, Russ; KAASHOEK, Frans; MORRIS, Robert. **xv6: a simple, Unix-like teaching operating system**. MIT, 2024. Disponível em: <https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf>. Acesso em: 30 maio 2026.

Documentação do XV6 no GitHub: MIT PDOS. **xv6-riscv**. [S. l.]: MIT, 2024. Disponível em: <https://github.com/mit-pdos/xv6-riscv>. Acesso em: 30 maio 2026.

AMINTAS VICTOR, [pseudônimo marcusvlc]. xv6. Versão: master. GitHub, 2021. Disponível em: <https://github.com/marcusvlc/xv6/tree/master>. Acesso em: 9 jul. 2026.